We implemented GitHub and file handling as our two advanced topics for our game Battleship.

We chose GitHub as our first topic to make the development of the game easier to manage between two people. We needed a way to work on the project without losing or overwriting each other's work.

We learned the basic concepts of GitHub, including creating a repository and sharing it with each other, creating new branches to organise our work, uploading changes and reviewing previous versions.

It has been really helpful, as we were able to work on the same project from two different computers. Every change that we made to it was easily uploaded and stored with a personalised message explaining the modifications. When working separately, this allowed us to know exactly what modifications the other partner had made. We could then download the latest version to keep our local projects synchronised.

We have learned the importance of maintaining effective and organised communication with our partner and regularly committing changes to avoid conflicts between different versions of the code. One of our main challenges was working on the same file simultaneously as we were both changing the code at the same time. However, thanks to GitHub, we were able to organise the modifications and carefully check them before combining our work.

The link to our repository is here: https://github.com/Mike-code-now/SinkTheFleet.git

We chose file handling as our second topic because we wanted to create a leaderboard for our Battleships game to make it more competitive and engaging, similar to an arcade machine. We needed a way to store players' wins so that they would not disappear when the game was closed and could be accessed again when restarting it.

We learned how to read and write text files in Java, store and retrieve information, and split strings to store the data into arrays.

We created a text file containing the players' names and scores. When a game finishes, the leaderboard is updated. If the player already exists, the whole file is rewritten with the player's wins updated; if it is a new player, their name and score are added to the file. This allows the leaderboard to be saved between sessions.

This allowed us to store data permanently instead of temporarily in variables that only retain information within the opened session. Our main challenge was deciding how to organise the information in the text file to make the leaderboard as efficient as possible. Initially, we considered storing only the names of the winners and counting how many times each name appeared to calculate their scores. However,

we decided that it would be more efficient to store each player's name alongside their score and simply update the score whenever they won another game.

Both topics were great additions to our program and helped us develop valuable programming skills that we will be able to apply in future projects.